

Timing-Driven DSP Placement for Systolic Arrays on FPGAs

Anonymous Author(s)

Anonymous Institution

Anonymous City, Anonymous Country

anonymous@anonymous.edu

Abstract

Systolic-array-based field-programmable gate array (FPGA) accelerators exhibit strong spatial regularity, yet their timing quality remains highly sensitive to placement. Existing FPGA placers are largely general-purpose. Our previous R-SAD-based flow identified an order-consistency (OC) structural viewpoint for systolic-array placement [1]. In this work, we develop a timing-driven digital signal processing (DSP) placement method that builds on this structural foundation. We first justify the use of OC from the half-perimeter wirelength (HPWL) viewpoint, and then show that the resulting weighted multi-net placement objective can be reduced to a shortest-path problem on a state directed acyclic graph (DAG). This yields an exact dynamic-programming (DP) solver for the fixed-weight subproblem and enables iterative timing-driven reweighting. Averaged over seven benchmarks, the proposed flow improves maximum frequency ($F_{\max}$) by 7.4% over Vivado and by 4.8% over R-SAD, while reducing HPWL by 24.0% relative to Vivado and incurring only a 1.9% increase relative to R-SAD. These results show that explicitly controlling the longest wirelength improves worst negative slack (WNS) and $F_{\max}$ while preserving essentially the same HPWL quality as the HPWL-oriented predecessor.

CCS Concepts

• **Hardware** → **Reconfigurable logic and FPGAs.**

Keywords

FPGA placement, systolic arrays, DSP placement, timing-driven placement, dynamic programming

1 Introduction

Systolic arrays are a dominant compute substrate in machine-learning (ML) acceleration on field-programmable gate arrays (FPGAs). Their implementations are typically digital signal processing (DSP)-heavy and structurally regular. However, this regularity does not by itself guarantee good timing. Once mapped to a heterogeneous FPGA fabric, a systolic array is strongly affected by long inter-processing-element (PE) connections, so placement has a direct impact on worst negative slack (WNS) and maximum frequency ($F_{\max}$).

This setting exposes a clear mismatch between generic FPGA placement and systolic-array placement. General-purpose placers do not explicitly exploit the structural regularity of the array. Our previous work, centered on the R-SAD algorithm, addressed this issue by introducing an order-consistency (OC) structural viewpoint for systolic-array placement and showing that this viewpoint can lead to strong half-perimeter wirelength (HPWL) results [1]. The present work builds on that foundation. Its goal is not to replace

R-SAD as an HPWL-oriented predecessor, but to extend the same structural line to the timing-driven setting.

The key difficulty is that timing-driven placement is not a single-metric HPWL problem. On FPGAs, timing is strongly influenced by the longest critical wires, so improving $F_{\max}$ requires more than reducing aggregate wirelength alone. The method therefore must optimize multiple nets under nonuniform timing-dependent weights while preserving the structural advantages identified by R-SAD.

The main contribution of this paper is a formulation result. We first justify the OC condition from the HPWL viewpoint, and then show that the resulting weighted multi-net objective can be reduced to the optimization of one shortest path on a state DAG. This converts the strip-placement problem into a dynamic-programming problem with exact optimal substructure for fixed weights. It also explains why the R-SAD line is scientifically meaningful: once order consistency is justified, the single-column placement problem admits exact optimization rather than only heuristic search.

Based on this formulation, we build a timing-driven placement flow that iteratively adjusts net weights according to timing pressure and repeatedly invokes the DP solver. On the seven benchmarks in our current study, the resulting flow improves average $F_{\max}$ by 7.4% over Vivado and by 4.8% over our previous R-SAD-based method. At the same time, it reduces average HPWL by 24.0% relative to Vivado, while increasing HPWL by only 1.9% relative to R-SAD. It also improves average WNS and total negative slack (TNS) over both baselines. These numbers support the central claim of this paper: explicitly controlling the longest wirelength is an effective path toward reducing WNS and improving $F_{\max}$ for systolic-array DSP placement, while preserving nearly the same HPWL quality as R-SAD.

The contributions of this paper are summarized as follows.

- We further clarify the role of the OC condition introduced by R-SAD and show how it turns the single-column DSP placement objective into an ordered form that is compatible with exact shortest-path optimization.
- We formulate timing-driven DSP placement for systolic arrays on FPGAs through a weighted multi-net objective and show that it can be converted into the shortest-path optimization of a state DAG.
- We develop a DP solver that is applicable to different net-weight settings, which makes it suitable for timing-driven reweighting rather than only one fixed deterministic placement policy.
- We build a timing-driven placement flow that improves average $F_{\max}$ by 7.4% over Vivado and 4.8% over the previous R-SAD-based method, while reducing average HPWL by 24.0% versus Vivado and incurring only a 1.9% HPWL increase over R-SAD.

The rest of the paper is organized as follows. Section 2 reviews the most relevant background. Section 3 introduces the problem

setting and notation. Section 4 presents the multi-column decomposition, the state-DAG formulation, and the timing-driven flow. Section 5 describes the experimental setup and results. Section 6 concludes the paper.

2 Related Work

2.1 Systolic-Array-Specific Placement

Our previous work on systolic-array placement on FPGAs established that exploiting array regularity can substantially improve placement quality over generic FPGA placement flows [1]. That work introduced the R-SAD algorithm together with the order-consistency (OC) condition. OC is the most relevant predecessor idea for the present paper: it states that the spatial order of processing elements should follow their array order, thereby turning single-column placement into a structured ordered problem rather than an unconstrained generic placement problem. The R-SAD line also motivates the region-wise decomposition used in this paper. In particular, for tall single-column strips, the central row-sweep (CTR) region serves as a structural pattern that can be exploited for acceleration by reducing an $m \times h$ problem to an $h \times h$ core problem before reconstruction.

More recently, SysMix extended the same research line toward mixed-size placement for hierarchical systolic-array designs [2]. The key message from these works is that systolic-array placement benefits from explicit structural modeling. However, they do not address the weighted shortest-path formulation developed here for timing-driven placement.

2.2 Timing-Driven FPGA Placement

Timing-driven placement for FPGAs has been studied from several angles, including heterogeneity-aware timing modeling and critical-path-oriented optimization [3, 4]. Prior work has shown that optimizing wirelength alone is often insufficient for timing closure and that timing-aware modeling must enter the placement flow explicitly. Our work is aligned with this direction, but it targets the narrower setting of DSP placement for systolic arrays and emphasizes a proof-oriented weighted dynamic-programming (DP) core rather than a general heterogeneous FPGA timing model. The technical distinction from R-SAD is not that R-SAD lacks value, but that R-SAD mainly optimizes the OC-constrained HPWL objective whereas the present work explicitly trades off aggregate wirelength and longest-wire timing pressure.

3 Problem Formulation

We consider digital signal processing (DSP) placement for an $m \times h$ systolic-array kernel on a field-programmable gate array (FPGA) architecture with sparse DSP columns. Here m denotes the number of array rows and h denotes the number of array columns within one strip. The basic single-column placement variable is an order matrix $y \in \{1, \dots, mh\}^{m \times h}$, where $y_{i,j}$ is the placement order assigned to the processing element at row i and column j . At this point, we do not yet impose the order-consistency (OC) condition; the role of OC will be justified in Section 4 before it is used algorithmically.

Under this representation, horizontal and vertical neighbor nets induce a half-perimeter wirelength (HPWL) objective on the order matrix. Let $w_{i,j}^H$ be the weight of the horizontal edge between (i, j)

and $(i, j + 1)$, and let $w_{i,j}^V$ be the weight of the vertical edge between (i, j) and $(i + 1, j)$. The weighted objective is

$$\min \sum_{i=1}^m \sum_{j=1}^{h-1} w_{i,j}^H |y_{i,j+1} - y_{i,j}| + \sum_{i=1}^{m-1} \sum_{j=1}^h w_{i,j}^V d_V |y_{i+1,j} - y_{i,j}|, \quad (1)$$

where d_V is the vertical spacing factor between adjacent rows.

This weighted formulation is essential for timing-driven placement. The weights are not fixed constants; they are updated according to timing pressure so that long or critical connections receive more emphasis in later rounds. In our flow, the practical timing signal is derived from limiting the longest wirelength. Let L_e denote the geometric length of edge e , and let T denote the target maximum wirelength. The slack-like quantity is

$$s_e = T - L_e. \quad (2)$$

Reducing the longest wirelength improves the worst edges first, which empirically leads to lower WNS and higher $F_{\max}$.

The challenge is therefore threefold. First, one must justify why it is legitimate to restrict attention to the OC-structured solution space introduced by R-SAD. Second, the solver must optimize Eq. (1) exactly for a given set of weights once this structure is adopted. Third, it must remain applicable when the weights change across timing-driven rounds. The key steps of this paper are to justify the use of OC from the HPWL viewpoint and then show that the OC-structured weighted objective can be converted into a shortest-path problem on a state DAG.

4 Proposed Method

4.1 Overview

The method has five layers. First, a multi-column DSP placement instance is decomposed into several single-column strip subproblems following the R-SAD strategy. Second, we justify the use of the order-consistency (OC) condition from the HPWL viewpoint. Third, under OC, the weighted HPWL objective is converted into a shortest-path problem on a state DAG and solved exactly by dynamic programming. Fourth, when the strip is tall, a CTR-based reduction shrinks an $m \times h$ instance to an $h \times h$ core problem and then expands the solution back. Finally, an outer timing-driven loop updates net weights according to longest-wire violations and repeatedly invokes the exact strip solver.

4.2 Multi-Column Decomposition

Our exact solver is built for single-column strip placement, so a multi-column FPGA placement must first be decomposed into strip subproblems. Here we deliberately follow the R-SAD strategy instead of introducing a new decomposition principle. Candidate column cuts are evaluated by estimating the wirelength range they can induce. The R-SAD analytical wirelength formula provides a lower bound for a given cut pattern, while the row-sweep construction provides a corresponding upper bound. These bounds are used to estimate the attainable wirelength range of each strip type and to choose the most promising column partition.

After a column partition is fixed, each strip is treated as a single-column placement problem. The contribution of the present paper

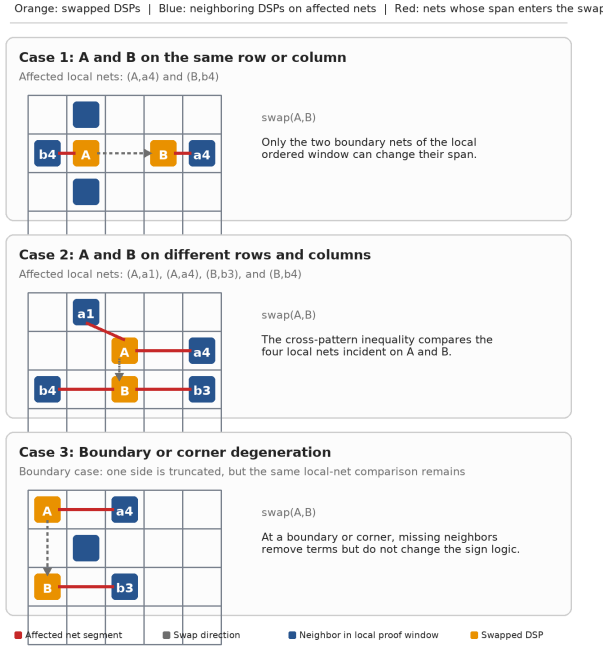


Figure 1: Local swap patterns and affected nets in the OC proof sketch. The three panels correspond to the same-row or same-column case, the different-row and different-column case, and a boundary or corner degeneration. Red segments denote the local nets whose span enters the swap-local HPWL comparison, while blue blocks mark the neighboring DSPs referenced by the proof.

therefore begins after the decomposition step: the column partitioning remains R-SAD-style, while the new technical content lies in the exact weighted strip solver and its timing-driven extension.

4.3 Why the OC Condition Is Legitimate

The first theoretical question is whether it is legitimate to restrict attention to the OC-structured solution space introduced by R-SAD.

Proposition 1. If a single-column placement is optimal under OC, then swapping any pair of DSP macros cannot decrease HPWL. Equivalently, an OC-optimal solution is locally optimal with respect to arbitrary two-swaps.

The proof sketch is direct and does not require a separate induction on swap distance. Consider an arbitrary two-swap between macros a and b . If the swap preserves OC, then HPWL cannot decrease because the original solution is already optimal within the OC-feasible set. The nontrivial case is when the swap breaks OC. In that case, all relative layouts can be reduced to three basic positional patterns:

- the two macros lie on the same row or the same column,
- the two macros lie on different rows and different columns, and
- boundary or corner configurations, which are degenerate forms of the previous two patterns.

For each pattern, the incident nets are partitioned into three groups: nets that are forced to become longer after the swap, nets whose contribution is unchanged, and nets that may become shorter. The key observation is that the forced increases always dominate the possible decreases, so the total HPWL change is nonnegative. Figure 1 provides an intuitive view of the three local swap patterns. Two algebraic motifs summarize the calculation.

Let $Y(\cdot)$ denote the row or column coordinate used in the corresponding local comparison window. In other words, $Y(a)$, $Y(a_1)$, and related terms denote the ordered coordinate values that appear in the swap-local HPWL calculation for the pattern under discussion.

For the same-row or same-column pattern,

$$\Delta\text{HPWL} = 2(Y(a_4) - Y(b_4)) \geq 0, \quad (3)$$

where the neighboring terms appear in matched pairs and the swap can only preserve or increase the total span.

For the different-row/different-column pattern,

$$\Delta\text{HPWL} = 2(Y(a_4) + Y(a_1) - Y(b_3) - Y(b_4)) \geq 0. \quad (4)$$

This is the core cross-pattern inequality in the slide proof. Boundary and corner cases do not require separate principles; they reduce to the same two algebraic comparisons together with strictly positive contributions from nearby nets that are forced to lengthen after the swap.

Therefore, for any pair of DSP macros and for any swap distance, a two-swap applied to an OC-optimal solution cannot decrease HPWL. Proposition 1 follows.

This is the scientific role of R-SAD in the present paper. OC is not introduced here as a heuristic convenience. Rather, it identifies a structured solution space that remains consistent with HPWL optimality in the single-column setting. Only after this justification do we impose OC and use it as the basis of the exact optimization framework below.

4.4 From OC to an Ordered Objective

After the above justification, we impose the order-consistency (OC) condition

$$y_{i,j} \leq y_{i,j+1}, \quad y_{i,j} \leq y_{i+1,j}, \quad (5)$$

which is exactly the monotonicity structure introduced by R-SAD.

Under OC, the relative order of every macro pair in a net is fixed. Therefore, each term $|y_{i_a,j_a} - y_{i_b,j_b}|$ in Eq. (1) has a determined sign and can be rewritten without the absolute value. Summing over all nets yields an ordered linear objective

$$\sum_{i,j} C_{i,j} y_{i,j}, \quad (6)$$

where $C_{i,j}$ aggregates the signed contributions of all incident nets to location (i, j) . This is the key algebraic step: once OC fixes pairwise ordering, the HPWL objective is transformed from an unordered absolute-value objective into an ordered linear objective over placement labels.

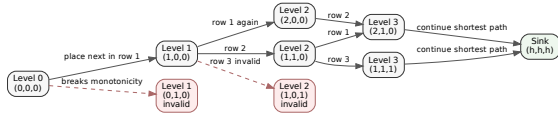


Figure 2: State-DAG view of the OC-constrained strip placement problem. A state records the monotone placement frontier, and each edge places exactly one new macro.

4.5 From Ordered Objective to State DAG

Eq. (6) can be further rewritten into an incremental form indexed by placement order. For each location (i, j) , define

$$\phi_{i,j} = d_V \left(\underbrace{w_{i-1,j}^V}_{\text{up}} + \underbrace{w_{i,j-1}^H}_{\text{left}} - \underbrace{w_{i,j}^V}_{\text{down}} - \underbrace{w_{i,j}^H}_{\text{right}} \right), \quad (7)$$

with out-of-range terms treated as zero. Here $\phi_{i,j}$ is the local coefficient of location (i, j) in the ordered objective. If order label $\ell + 1$ is assigned to location (i, j) , the incremental contribution is

$$c_\ell(i, j) = (\ell + 1) \phi_{i,j}. \quad (8)$$

Thus the total objective can be accumulated one placement step at a time.

We now define the state space. At level ℓ , a state is

$$a = (a_1, \dots, a_m), \quad (9)$$

where $a_i \in \{0, \dots, h\}$ is the number of already placed macros in row i , and $\ell = \sum_i a_i$ is the number of already assigned labels. Under OC, the frontier must remain monotone:

$$a_1 \geq a_2 \geq \dots \geq a_m. \quad (10)$$

The source state is $(0, \dots, 0)$ and the sink state is $(h, \dots, h)$.

An edge in the DAG corresponds to incrementing exactly one row count. If state a' is obtained from a by increasing a_i by one, then the next order label is assigned to location $(i, a_i + 1)$. The move is legal only if the frontier remains monotone. Therefore each source-to-sink path corresponds to one feasible OC placement, and the path cost is the sum of all incremental costs. Let $\mathcal{P}(s, t)$ denote the set of all source-to-sink paths in the state DAG. The original optimization is reduced to

$$\min_{P \in \mathcal{P}(s,t)} \sum_{(a,a') \in P} c(a, a'), \quad (11)$$

which is a shortest-path problem on a DAG.

Key contribution. The main theoretical contribution of this paper is exactly this conversion from optimizing many weighted net HPWL terms to optimizing one shortest path on a state DAG.

4.6 Dynamic Programming Solver

Let $f_\ell(a)$ denote the minimum cost of reaching state a after placing the first ℓ labels. Since the graph is acyclic, the DP recurrence is

$$f_{\ell+1}(a') = \min_{a \rightarrow a'} (f_\ell(a) + c(a, a')). \quad (12)$$

The optimal placement is recovered by tracing the minimum-cost source-to-sink path. A forward-backward reconstruction scheme can be used to avoid storing the full DP table.

4.7 CTR-Based Acceleration

The exact DP solver is expensive on tall strips. A key acceleration is the central row-sweep (CTR) observation inherited from the R-SAD line. When mapping an $m \times h$ array to one DSP column and $m > h > 6$, the middle part of the optimal placement exhibits a row-sweep pattern, called the CTR region in R-SAD. What begins as an empirical observation can be converted into an algorithmic reduction.

Instead of solving the full $m \times h$ strip directly, we shrink it to an $h \times h$ core problem, solve that core exactly, and then expand the solution back to the original height. Concretely, the top and bottom bands are preserved from the solved square core, while the middle rows are filled by a row-sweep CTR band. In effect, an $m \times h$ mapping problem is converted into an $h \times h$ problem, and the resulting square solution is split into top and bottom parts with a row-sweep region inserted in the middle. This substantially reduces the computational burden while preserving the structural quality of the placement.

4.8 Timing-Driven Weight Update

The exact strip solver is placed inside an outer timing-driven loop. Starting from initial weights, the solver produces a weighted OC placement. That placement is then analyzed to obtain the longest edge, WNS, and TNS under the target T .

For each edge e , let L_e denote its current geometric length. The slack-like quantity is

$$s_e = T - L_e. \quad (13)$$

If all tracked edges satisfy $s_e \geq 0$, the current strip already satisfies the target. Otherwise, the most critical edges receive larger weights.

The weight update borrows the net-weight adaptation style used in DreamPlace 4.0 [7]. The solver keeps log-weights and momentum buffers:

$$\Delta \log w_e^{(t+1)} = \alpha \Delta \log w_e^{(t)} + \eta \log(1 + c_e^{(t)}), \quad (14)$$

$$\log w_e^{(t+1)} = \log w_e^{(t)} + \Delta \log w_e^{(t+1)}. \quad (15)$$

Here $c_e^{(t)}$ is derived from the current violation magnitude, and only the most critical edges are strengthened in each round.

This creates a clean division of labor:

- the inner DP layer solves the weighted OC strip-placement problem exactly for fixed weights, and
- the outer loop changes those weights to reduce the longest wires and improve timing.

This also explains the empirical tradeoff against R-SAD. R-SAD is highly effective when the objective is static HPWL minimization under OC. Our method keeps that OC structure and near-optimal HPWL behavior, but additionally drives down the longest wires, which yields better WNS and $F_{\max}$ at the cost of only a very small HPWL increase.

4.9 Discussion of Optimality Scope

The exact optimality claim of this paper applies to the weighted state-DAG subproblem for a fixed set of net weights. The full timing-driven flow remains iterative because weights change between solves. Accordingly, the paper should claim:

Table 1: Average results over seven benchmarks.

Method	HPWL	WNS	TNS	$F_{\max}$
Vivado	683,314	-0.71	-1311.67	231.91
R-SAD	509,232	-0.61	-1245.90	237.70
AMFPlace2.0 + Vivado P&R	927,999	-4.99	-12837.49	117.46
DSPlace + Vivado P&R	746,915	-1.67	-5634.44	191.72
Ours	519,018	-0.39	-952.81	249.08

- exact shortest-path optimality for the fixed-weight OC subproblem, and
- empirical timing improvement for the full timing-driven outer loop.

5 Experimental Setup and Results

5.1 Setup

The current comparison table reports seven benchmarks and five methods. The experiments compare the proposed timing-driven dynamic-programming (DP) flow against Vivado, R-SAD, AMFPlace 2.0 + Vivado P&R, and DSPlace + Vivado P&R. The target audience should read the first two comparisons differently:

- Vivado is the industrial baseline.
- R-SAD is the direct predecessor and isolates the value of the new timing-driven formulation over the prior OC-based HPWL-oriented deterministic method.

This distinction is important. Relative to Vivado, the paper should emphasize that problem-specific systolic-array placement already has large value. Relative to R-SAD, the paper should emphasize that the new contribution is not better HPWL minimization alone, but a better timing tradeoff through explicit control of the longest wirelength.

5.2 Main Results

The current quantitative outcomes are already sufficient to anchor the paper’s central story. Averaged over the seven benchmarks, the proposed flow achieves 249.08 MHz $F_{\max}$, compared with 231.91 MHz for Vivado and 237.70 MHz for R-SAD. This corresponds to a 7.4% $F_{\max}$ gain over Vivado and a 4.8% gain over R-SAD. In terms of wirelength, the proposed flow reduces average HPWL from 683,314 to 519,018 relative to Vivado, while remaining very close to R-SAD (509,232 versus 519,018, only +1.9%).

The timing metrics are consistent with the same story. Compared with Vivado, the proposed flow reduces the magnitude of average worst negative slack (WNS) from 0.71 to 0.39 and the magnitude of average total negative slack (TNS) from 1311.67 to 952.81. Compared with R-SAD, it reduces the magnitude of average WNS from 0.61 to 0.39 and the magnitude of average TNS from 1245.90 to 952.81. These numbers suggest two points. First, the timing-driven objective is effective in moving the design toward better timing, which is consistent with the intended longest-wirelength control path. Second, the small HPWL regression against R-SAD indicates that the new method is not simply a wirelength minimizer; it is trading a very small amount of aggregate wirelength for stronger timing quality.

5.3 Interpretation

The paper should interpret these results through the mechanism, not only through the percentages. Tables 2 and 3 show that the proposed method is the most timing-effective approach among all five compared methods: it delivers the best average $F_{\max}$ and the best average timing-slack metrics, while keeping HPWL close to R-SAD and substantially better than the three placement-and-route baselines built around generic flows. This is exactly the behavior expected from the method. Explicitly limiting the longest wirelength is a practical path to lowering WNS and improving $F_{\max}$. This is why $F_{\max}$ should remain the first metric in the result discussion, while HPWL serves as the supporting tradeoff metric. The role of R-SAD should also be stated carefully: R-SAD remains the stronger HPWL-oriented predecessor because it exploits the OC structure effectively, and our method wins by adding timing awareness on top of that same structural foundation.

5.4 Pending Insertions

The next revision should add:

- one ablation on the timing-driven weight update,
- one case-study figure showing a representative placement effect, and
- a short paragraph discussing why R-SAD can still be better in pure HPWL on some cases while losing in $F_{\max}$.

6 Conclusion

This paper presents a timing-driven DSP placement method for systolic arrays on FPGAs. The central contribution is a dynamic-programming formulation that converts the optimization of multiple weighted net HPWL terms into the shortest-path optimization of a state DAG. This formulation builds on the OC structure introduced by R-SAD, clarifies why that structure is valuable for HPWL optimization, and extends it to the weighted timing-driven setting. Experimental results on seven benchmarks show that the resulting flow improves average $F_{\max}$ over both Vivado and the previous R-SAD-based method while keeping HPWL very close to R-SAD. These results support the view that controlling the longest wirelength is an effective path toward lowering WNS and improving timing quality in systolic-array DSP placement without sacrificing the structural advantages of the OC-based formulation.

References

- [1] H. Hu, D. Fang, W. Li, B. Yuan, and J. Hu, “Systolic Array Placement on FPGAs,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023.
- [2] D. Fang, H. Hu, W. Li, B. Yuan, and J. Hu, “SysMix: Mixed-Size Placement for Systolic-Array-Based Hierarchical Designs,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2024.
- [3] Z. Lin, Y. Xie, G. Qian, J. Chen, S. Wang, J. Yu, and Y.-W. Chang, “Timing-Driven Placement for FPGAs with Heterogeneous Architectures and Clock Constraints,” in *Proc. Design, Automation and Test in Europe (DATE)*, 2021.
- [4] S. Dhar, M. A. Iyer, S. Adya, L. Singhal, N. Rubanov, and D. Z. Pan, “An Effective Timing-Driven Detailed Placement Algorithm for FPGAs,” in *Proc. Int. Symp. Physical Design (ISPD)*, 2017.
- [5] J. Pui, S. Choi, A. Chowdhury, W. Li, S. Tang, D. Z. Pan, Y. Ye, and X. Yang, “RippleFPGA: A Routability-Driven Placement for Large-Scale Heterogeneous FPGAs,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2016.
- [6] Y. Li, B. Liao, C. Weng, and D. Z. Pan, “UTPlaceF 3.0: A Parallelization Framework for Modern FPGA Global Placement,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2017.

Table 2: Benchmark-wise HPWL and $F_{\max}$ comparison for all five methods.

Case	Vivado		R-SAD		AMFPlace 2.0 + Vivado P&R		DSPlace + Vivado P&R		Ours	
	HPWL	$F_{\max}$	HPWL	$F_{\max}$	HPWL	$F_{\max}$	HPWL	$F_{\max}$	HPWL	$F_{\max}$
OS24x24_W8	245232	263.71	177188	261.99	282005	136.91	292413	241.25	177441	262.54
OS32x32_W8	494792	239.06	356790	222.52	628927	121.86	543872	187.62	376203	244.92
OS36x32_W8	555410	202.51	568842	246.61	908625	116.55	650782	194.67	578173	247.71
OS40x32_W8	690842	230.41	448668	198.41	969454	124.35	781080	180.57	475704	244.56
OS40x36_W8	774885	238.55	540131	246.79	993061	116.89	880601	173.22	539512	250.50
OS40x40_W8	854961	233.26	597408	245.34	1704262	104.01	982803	184.26	593826	248.45
OS44x40_W8	1167074	215.84	875596	242.25	1009659	101.68	1096855	180.44	892266	244.86
AVG	683314	231.91	509232	237.70	927999	117.46	746915	191.72	519018	249.08

Table 3: Benchmark-wise WNS and TNS comparison for all five methods.

Case	Vivado		R-SAD		AMFPlace 2.0 + Vivado P&R		DSPlace + Vivado P&R		Ours	
	WNS	TNS	WNS	TNS	WNS	TNS	WNS	TNS	WNS	TNS
OS24x24_W8	-0.29	-273.47	-0.32	-297.27	-3.70	-4584.19	-0.55	-485.23	-0.31	-351.68
OS32x32_W8	-0.58	-1298.78	-0.89	-2289.01	-4.61	-9662.71	-1.73	-4358.38	-0.48	-1455.69
OS36x32_W8	-1.34	-1537.04	-0.46	-1002.22	-4.98	-12272.79	-1.54	-4260.06	-0.44	-988.01
OS40x32_W8	-0.74	-1564.21	-1.44	-2285.96	-4.44	-12776.95	-1.94	-6675.55	-0.49	-1239.85
OS40x36_W8	-0.59	-1442.15	-0.45	-1115.90	-4.96	-13466.69	-2.17	-7940.15	-0.39	-1055.06
OS40x40_W8	-0.39	-39.24	-0.18	-22.46	-6.01	-20995.48	-1.83	-6782.90	-0.13	-12.35
OS44x40_W8	-1.03	-3026.82	-0.53	-1708.49	-6.24	-16103.59	-1.94	-8938.83	-0.48	-1567.06
AVG	-0.71	-1311.67	-0.61	-1245.90	-4.99	-12837.49	-1.67	-5634.44	-0.39	-952.81

- [7] P. Liao, D. Guo, Z. Guo, S. Liu, Y. Lin, and B. Yu, "DREAMPlace 4.0: Timing-Driven Placement With Momentum-Based Net Weighting and Lagrangian-Based Refinement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 10, pp. 3374–3387, 2023.